

کامپایلر NNGraph DSL

یک زبان دامنه‌محور برای توصیف معماری شبکه‌های عصبی
و تولید خودکار کد PyTorch

گزارش فنی پروژه

۱۱ تیر ۱۴۰۵

فهرست مطالب

۲	چکیده
۳	۱ مقدمه
۳	۱.۱ انگیزه
۴	۲.۱ هدف
۴	۳.۱ دامنه کاربرد
۵	۲ کارهای مرتبط
۵	۱.۲ چارچوب NADER
۵	۲.۲ FANTLR
۵	۱.۲.۲ الگوی Listener در برابر Visitor
۶	۳.۲ ابزارها و تجسم‌سازهای AST
۶	۱.۳.۲ چرا AST در اینجا استفاده نمی‌شود
۸	۳ طراحی زبان
۸	۱.۳ ساختار برنامه
۹	۲.۳ گرامر
۱۰	۳.۳ سیستم انواع
۱۱	۴.۳ انواع لایه
۱۱	۱.۴.۳ لایه‌های پارامتردار
۱۱	۲.۴.۳ فعال‌سازی‌ها
۱۱	۳.۴.۳ عملیات گراف (بدون ماژول)
۱۲	۴.۴.۳ پارامترهای سراسری
۱۲	۵.۳ نحو یال
۱۳	۶.۳ مثال کامل: MLP
۱۵	۴ پیاده‌سازی
۱۵	۱.۴ معماری کامپایلر

۱۶	مسئولیت فایل‌ها	۱.۱.۴
۱۶	تحلیل‌گر معنایی	۲.۴
۱۶	انتشار مقدار در درخت	۱.۲.۴
۱۷	قوانین اعتبارسنجی	۲.۲.۴
۱۷	قالب پیام خطا	۳.۲.۴
۱۸	FalseCtx_ برای خطاهای پس از پیمایش	۴.۲.۴
۱۸	جریان اجرای شنونده	۵.۲.۴
۲۲	مولد کد	۳.۴
۲۳	الگوریتم تخصیص متغیر	۴.۴
۲۳	ردیابی MLP (زنجیره خطی)	۱.۴.۴
۲۴	ردیابی ResBlock (فناوت سپس ادغام)	۲.۴.۴
۲۴	ردیابی InceptionBlock (دو شاخه موازی)	۳.۴.۴
۲۵	ردیابی TransformerEncoder (دو residual ضمنی)	۴.۴.۴
۲۶	موارد خاص تولید کد	۵.۴.۴
۲۷	ساختار کد خروجی	۶.۴.۴
۲۷	استنتاج شکل	۵.۴
۲۸	تجسم Graphviz	۶.۴
۲۸	داربست آموزش	۷.۴
۲۸	حاشیه‌نویسی کوانتیزاسیون	۸.۴
۲۸	صادرکردن ONNX	۹.۴

۵ ارزیابی

۲۹	مجموعه آزمایش	۱.۵
۳۰	نتایج pass forward	۲.۵
۳۰	اثبات درستی متغیرها	۳.۵
۳۱	آمار کدبیس	۴.۵

۶ تصمیمات طراحی

۳۲	Listener در برابر Visitor	۱.۶
۳۲	load_from_listener() در برابر generate(traversal)	۲.۶
۳۲	نام‌گذاری متغیر با node_id	۳.۶

۷ نتیجه‌گیری و کارهای آینده

۳۳	نتیجه‌گیری	۱.۷
۳۳	قابلیت‌های پیاده‌سازی‌شده	۲.۷
۳۴	کارهای آینده	۳.۷

۳۵

منابع

چکیده

راستش NNGraph DSL یه زبان دامنه‌محور باحاله که ما برای توصیف معماری شبکه‌های عصبی به شکل گراف جهت‌دار طراحی کردیم فایل‌های `.nng` رو می‌دیم بهش و اونم برامون کد اجرایی PyTorch تولید می‌کنه که یه زیرکلاس `nn.Module` تر و تمیز با متدهای `__init__` و `forward` می‌ده بیرون

برای کامپایلر رفتیم سراغ ANTLR۴ نسخه ۲.۱۳.۴ و از الگوی Listener برای تحلیل معنایی استفاده کردیم یه الگوریتم مرتب‌سازی توپولوژیک Kahn هم زدیم تنگش که هم تو تحلیل‌گر معنایی (واسه تشخیص دور) و هم تو تولید کد (واسه ترتیب اجرا) حسابی به کارمون میاد تازه استنتاج شکل تنسور، کشیدن گراف با Graphviz، داربست آموزش، حاشیه‌نویسی کوانتیزاسیون و خروجی ONNX رو هم بهش اضافه کردیم تو بخش تست هم ۵۵ تا تست تو چهار تا فایل نوشتیم چهار تا معماری باحال مثل MLP، ResBlock، InceptionBlock و TransformerEncoder رو هم کامپایل کردیم و `forward pass` شون رو با شکل خروجی درست اجرا کردیم

فصل ۱

مقدمه

۱.۱ انگیزه

نوشتن زیرکلاس‌های `nn.Module` تو `PyTorch` واسه معماری‌های پیچیده خیلی رو اعصابه و کلی باگ میده مثلاً واسه یه شبکه ده لایه، ما باید:

- تو `__init__` هر لایه رو دستی با پارامترای خودش تعریف کنیم
 - تو `forward` هم ترتیب اجرا رو خودمون مدیریت کنیم
 - تو توپولوژی‌های شاخه‌دار هم باید واسه متغیرهای میانی اسم بذاریم
 - حواسمون باشه هیچ تنسور مشترکی قبل از اینکه استفاده بشه رو خودش بازنویسی نشه
- یه نگاه به این کد `PyTorch` بندازین که مجبوریم دستی بنویسیم:

قطعه کد ۱.۱.۱ > نمونه boilerplate دستی برای یک `ResBlock`

```
1 class ResBlock(nn.Module):
2     def __init__(self):
3         super(ResBlock, self).__init__()
4         self.conv1 = nn.Conv2d(
5             in_channels=64, out_channels=64,
6             kernel_size=3, stride=1, padding=1)
7         self.bn1 = nn.BatchNorm2d(num_features=64)
8         self.relu1 = nn.ReLU()
9         # ... and so on for each layer
```

```
10
11     def forward(self, x):
12         identity = x
13         x = self.conv1(x)
14         x = self.bn1(x)
15         x = self.relu1(x)
16         # ... manage residual connection manually
17         x = x + identity
18         return x
```

با NNGraph DSL، ما معماری رو به بار مثل یه گراف توصیف می‌کنیم و کامپایلر بقیه کدها رو برامون تولید می‌کنه

۲.۱ هدف

هدف ما از NNGraph DSL اینه که فایل `nng` بشه تنها جایی که معماری رو توش نگه می‌داریم اینطوری گراف رو توصیف می‌کنیم و کامپایلر کل اون کدهای تکراری رو خودش برامون می‌سازه

۳.۱ دامنه کاربرد

به نظرم این ابزار واسه معماری‌های ثابت خیلی جوابه، یعنی شبکه‌هایی که گراف محاسباتی‌شون همون اول کار معلومه البته واسه نسخه‌های بعدی تو برنامه‌مون هست که کنترل جریان پویا تو `forward` رو هم اضافه کنیم

فصل ۲

کارهای مرتبط

۱.۲ چارچوب NADER

تعریف ۱.۱.۲ > NADER

NADER (neural architecture design via representation) یک چارچوبه که ما باهاش معماری شبکه‌های عصبی رو به شکل یک گراف محاسباتی جهت‌دار (DAG) مدل می‌کنیم اینجا هر گره یک عملیات لایه‌ست و هر یال هم جریان تنسور رو نشون میده

ما تو NNGraph DSL همین مدل گراف رو پایه کارمون قرار دادیم

۲.۲ FANTLR

تعریف ۱.۲.۲ > FANTLR

ANTLR۴ (another tool for language recognition) یک ابزار خفنه که از رو فایل گرامر g4 . برامون lexer، parser و listener/visitor رو خودکار می‌سازه نسخه‌ای که ما استفاده کردیم ANTLR ۴.۱۳.۲ هست

۱.۲.۲ الگوی Listener در برابر Visitor

کلاً ANTLR۴ دو تا الگو واسه پیمایش درخت تجزیه داره:

• **listener**: اینجا ParseTreeWalker خودش درخت رو پیمایش می‌کنه و متدهای enter*/exit*

رو صدا می‌زنه نیازی هم نیست چیزی برگردونه

• **visitor**: تو این یکی ما خودمون پیمایش رو دست می‌گیریم و هر متدی یه چیزی برمی‌گردونه

ما تو این کامپایلر از الگوی Listener استفاده کردیم متدهای `exit*` تو فایل `custom_listener.py` وضعیت رو تو `self.nodes` و `self.edges` نگه می‌دارن، که به نظرم واسه ساختن گراف از رو درخت خیلی طبیعی‌تره

۳.۲ ابزارها و تجسم‌سازهای AST

واسه این کامپایلر ما از ابزارهای استاندارد درخت نحو انتزاعی (AST) واسه بازرسی و دیدن درخت استفاده کردیم کلاً سه تا فایل واسه این کار داریم:

• `ast.py` – کلاس AST و `TreeNode`

• `make_ast_subtree.py` – ترکیب گره‌های درخت تجزیه

• `ast_to_networkx_graph.py` – کشیدن گراف به صورت اختیاری

۱.۳.۲ چرا AST در اینجا استفاده نمی‌شود

بیشتر کامپایلر برنامه‌ها رو به صورت درخت نحو انتزاعی (AST) نشون می‌دن زبان‌هایی که رو عبارت‌ها می‌چرخن، خیلی راحت به ساختار درختی تبدیل می‌شن

اما داستان NNGraph DSL فرق داره ما معماری شبکه‌های عصبی رو مثل یه گراف جهت‌دار بدون دور (DAG) می‌بینیم گره‌ها همون لایه‌ها هستن و یال‌ها هم جریان تنسورها رو نشون می‌دن مدل ما اینجا خود گرافه، نه درخت تجزیه

واسه همین ما کلی عملیات گرافی انجام می‌دیم که به لیست مجاورت و یال‌ها نیاز دارن:

- **مرتب‌سازی توپولوژیک** (الگوریتم Kahn) واسه ترتیب اجرا تو `forward()`
- **تشخیص دور** با مرتب‌سازی توپولوژیک (واسه اینکه مطمئن بشیم DAG ما درسته)
- **تحلیل دسترس‌پذیری** با یه BFS مشتق از ورودی تا خروجی
- **اعتبارسنجی اتصال باقی‌مانده** (چک کردن درجه ورودی گره‌های ادغام)
- **استنتاج شکل** با انتشار شکل تنسورها رو یال‌ها
- **خروجی Graphviz DOT** واسه اینکه معماری رو قشنگ ببینیم
- **خروجی ONNX** (که خودش بر اساس فرمت گرافه)

یه AST فقط سلسله مراتب درخت تجزیه رو به ما می‌ده (`node_decl`, `graph_block`, `model_block`)، اما گراف محاسباتی که ما می‌خوایم رو قایم می‌کنه واسه همین تحلیل‌گر معنایی ما گره‌ها و یال‌ها رو مستقیم تو دیکشنری‌ها و لیست‌ها ذخیره می‌کنه که واسه الگوریتم‌های گراف خیلی بهتره

کدای ساخت AST (exitStart، exitEveryRule) تو custom_listener.py فقط
واسه دیباگ اختیاری با پرچم --show-ast اونجا هستن و موقع کامپایل اصلاً کاری به کارشون
نداریم

فصل ۳

طراحی زبان

۱.۳ ساختار برنامه

هر فایل nng . کلاً سه تا بلوک داره بلوک config هم که اختیاریه:

قطعه کد ۱.۱.۳ > ساختار کلی یک فایل nng.

```
1 model <Name> {
2     input  <id> : tensor(<shape>)
3     output <id>
4 }
5
6 graph {
7     node <id> : <LayerType>(<params>)
8     ...
9     edge <src> -> <dst>
10    edge <src> -> <dst> [label="<string>"]
11    ...
12 }
13
14 config {
15     batch_size = <int>
16     device      = "cpu" | "cuda"
```

```

17     loss      = "<LossFn>"          // enables training
    ↪ scaffold
18     optimizer = "<Optimizer>"      // Adam | SGD | AdamW |
    ↪ RMSprop
19     lr        = <float>             // learning rate
20     epochs    = <int>               // training epochs
21 }

```

تعریف ۲.۱.۳ > بلوک model

بلوک model اسم کلاس تولیدشده، شناسه گره ورودی و شکل تانسور ورودی، و شناسه گره خروجی رو مشخص می‌کنه

تعریف ۳.۱.۳ > بلوک graph

ما تو بلوک graph همه گره‌های لایه و یال‌های جهت‌دار رو تعریف می‌کنیم ترتیب نوشتن یال‌ها اصلاً مهم نیست چون کامپایلر خودش ترتیب اجرا رو از رو ساختار گراف درمیاره

تعریف ۴.۱.۳ > بلوک config

بلوک config واسه تعیین یه سری مقادیر تو بلوک __main__ کد تولیدشده به کار میره مثلاً batch_size پیش‌فرضش ۱ هست و device هم 'cpu' اگه کلید loss رو تنظیم کنیم، مولد کد برامون یه حلقه آموزش کامل با بهینه‌ساز و تابع خطا می‌سازه

۲.۳ گرامر

تعریف ۱.۲.۳ > آمار گرامر FNNGraph.g

- فایل گرامر: FNNGraph.g4 (۵۶ خط)
- قوانین تجزیه‌گر: ۱۴ تا قانون (snake_case)
- توکن‌های واژه‌نگار: ۲۰ تا توکن (ALL_CAPS)

قوانین کامل گرامر هم ایناست:

قطعه کد ۲.۲.۳ > گرامر NNGraph.g

```

1 grammar NNGraph;
2 start      : model_block graph_block config_block? EOF;
3 model_block : MODEL ID '{' input_decl output_decl '}';
4 input_decl  : INPUT ID ':' TENSOR shape_expr;
5 output_decl : OUTPUT ID;
6 graph_block : GRAPH '{' (node_decl | edge_decl)* '}';
7 node_decl   : NODE ID ':' layer_expr;
8 layer_expr  : ID '(' param_list? ')';
9 param_list  : param (',' param)*;
10 param       : ID '=' value;
11 edge_decl   : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
12 config_block : CONFIG '{' config_entry* '}';
13 config_entry : ID '=' value;
14 value       : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
15              | STRING | NONE | shape_expr;
16 shape_expr  : '(' INT_LITERAL (',' INT_LITERAL)* ')';

```

نکته مهم طراحی: واسه `input_decl` ما از `shape_expr` استفاده کردیم نه `shape_expr` چون `TENSOR '(' shape_expr ')'` خودش پرانتز داره اینطوری از یه باگ رایج که تو نسخه‌های اولیه‌مون داشتیم جلوگیری کردیم

۳.۳ سیستم انواع

مثال	نوع Python	نوع در DSL
128	int	int
1.0	float	float
false / true	bool	bool
"relu"	str	string
(64, 32, 32)	tuple[int]	shape
None	NoneType	None

ما تمایز بین `int` و `float` رو همون زمان کامپایل چک می‌کنیم مثلاً اگه بنویسیم `Dropout(p=1)`

بهمون ارور می‌ده چون p منتظر یه float هست نه int

۴.۳ انواع لایه

۱.۴.۳ لایه‌های پارامتردار

این لایه‌ها موقع ساخت `__init__` به شکل `self.<id> = nn.X(...)` تو کدمون جا می‌گیرن

کلیدواژه DSL	کلاس PyTorch	پارامترهای اصلی
Linear	<code>nn.Linear</code>	<code>bias, out_features, in_features</code>
Conv2d	<code>nn.Conv2d</code>	<code>,stride, kernel, out_ch, in_ch, padding</code>
Conv1d	<code>nn.Conv1d</code>	<code>stride, kernel, out_ch, in_ch</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>stride, kernel</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>stride, kernel</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>end_dim, start_dim</code>
Embedding	<code>nn.Embedding</code>	<code>embedding_dim, num_embeddings</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>num_heads, embed_dim</code>
LSTM	<code>nn.LSTM</code>	<code>,hidden_size, input_size, num_layers</code>
GRU	<code>nn.GRU</code>	<code>hidden_size, input_size</code>

۲.۴.۳ فعال‌سازی‌ها

`ELU(alpha)` و `LeakyReLU(negative_slope)`, `Softmax(dim)`, `GELU`, `Tanh`, `Sigmoid`, `ReLU`

۳.۴.۳ عملیات گراف (بدون ماژول)

اینا دیگه تو `__init__` خطی تولید نمی‌کنن:

کلیدواژه	پارامترها	کد تولیدشده
Add	–	out = a + b
Concat	dim: int	out = torch.cat([a,b], dim=d)
Residual	–	out = main + shortcut
Split	chunks: int, dim: int	split_0, split_1, ... = torch.chunk(x,n,dim=d)

۴.۴.۳ پارامترهای سراسری

یه پارامتر باحال به اسم quant داریم که رو هر گرهی می‌تونیم بذاریم مقداری که قبول می‌کنه ایناست: "dynamic"، "static" و "qat"

قطعه کد ۱.۴.۳ > نمونه استفاده از کوانتیزاسیون

```
1 node fc1 : Linear(in_features=784, out_features=128,
  ↳ quant="dynamic")
```

۵.۳ نحو یال

تعریف ۱.۵.۳ > یال ساده

```
edge src -> dst
```

اینطوری یه یال جهت‌دار از گره src به گره dst می‌سازیم

تعریف ۲.۵.۳ > یال برچسب‌دار

```
edge src -> dst [label="name"]
```

اینم واسه تعریف یه یال با برچسب رشته‌ایه ما کلاً برچسب‌ها رو واسه دو تا کار استفاده می‌کنیم:

- "shortcut": واسه اینکه یال اتصال کوتاه به گره (Residual) رو نشون بدیم
- "residual_*": واسه اتصال کوتاه به گرهی که دو تا ورودی داره (مثل الگوی (transformer

۶.۳ مثال کامل: MLP

فایل nng. واسه یه شبکه سه لایه این شکلی می شه:

قطعه کد ۱.۶.۳ > mlp.nng

```

1 model MLP {
2     input x : tensor(784)
3     output out
4 }
5
6 graph {
7     node fc1 : Linear(in_features=784, out_features=256)
8     node relu1 : ReLU()
9     node fc2 : Linear(in_features=256, out_features=128)
10    node relu2 : ReLU()
11    node drop : Dropout(p=0.3)
12    node fc3 : Linear(in_features=128, out_features=10)
13    node out : Softmax(dim=1)
14
15    edge x -> fc1
16    edge fc1 -> relu1
17    edge relu1 -> fc2
18    edge fc2 -> relu2
19    edge relu2 -> drop
20    edge drop -> fc3
21    edge fc3 -> out
22 }
23
24 config {
25     batch_size = 64
26     device = "cuda"
27 }
```

و اینم کد PyTorch که برامون تولید می کنه:

قطعه کد ۲.۶.۳ > کد PyTorch تولیدشده برای MLP

```

1  import torch
2  import torch.nn as nn
3
4
5  class MLP(nn.Module):
6      def __init__(self):
7          super(MLP, self).__init__()
8          self.fc1 = nn.Linear(in_features=784,
9                               ↪ out_features=256)
10         self.relu1 = nn.ReLU()
11         self.fc2 = nn.Linear(in_features=256,
12                               ↪ out_features=128)
13         self.relu2 = nn.ReLU()
14         self.drop = nn.Dropout(p=3.0)
15         self.fc3 = nn.Linear(in_features=128,
16                               ↪ out_features=10)
17         self.out = nn.Softmax(dim=1)
18
19     def forward(self, x):
20         x = self.fc1(x)
21         x = self.relu1(x)
22         x = self.fc2(x)
23         x = self.relu2(x)
24         x = self.drop(x)
25         x = self.fc3(x)
26         x = self.out(x)
27         return x
28
29 if __name__ == '__main__':
30     device = torch.device('cuda')
31     model = MLP().to(device)
32     x = torch.randn(64, 784).to(device)
33     print(model(x).shape)

```

فصل ۴

پیاده‌سازی

۱.۴ معماری کامپایلر

تعریف ۱.۱.۴ > خط لوله کامپایل

ما خط لوله کامپایلر رو تو شش تا مرحله چیدیم:

۱. تجزیه: `FileStream` → `NNGraphLexer` → `NNGraphParser.program()` →

درخت تجزیه

۲. تحلیل معنایی: `NNGraphCustomListener + ParseTreeWalker` → مدل گراف

۳. مرتب‌سازی توپولوژیک: اینجا با الگوریتم Kahn ترتیب اجرا رو درمیاریم

۴. استنتاج شکل: شکل تنسورها رو تو گراف پخش می‌کنیم و اگه مشکلی بود مچش رو می‌گیریم

(اختیاری: `-show-shapes`)

۵. تولید کد: `CodeGenerator.generate()` → رشته پایتونی

۶. خروجی: تو این مرحله می‌ریزیم تو فایل `.py` یا به خروجی `Graphviz DOT` می‌گیریم یا کلاً

خروجی `ONNX` می‌دیم بیرون

۱.۱.۴ مسئولیت فایل‌ها

فایل	نقش
NNGraph.g4	منبع حقیقت گرامر (۵۶ خط)
gen/NNGraphLexer.py	تولیدشده توسط ۴ANTLR
gen/NNGraphParser.py	تولیدشده توسط ۴ANTLR
gen/NNGraphListener.py	listener پایه – نباید ویرایش شود
custom_listener.py	تحلیل معنایی
code_generator.py	تولید کد PyTorch
shape_inference.py	استنتاج شکل تنسور در کامپایل
graph_viz.py	تولید نمودار DOT Graphviz
onnx_export.py	صادرکردن مدل به فرمت ONNX
main.py	رابط CLI و compile_nng()

۲.۴ تحلیل‌گر معنایی

تعریف ۱.۲.۴ > NNGraphCustomListener

ما به کلاس NNGraphCustomListener(NNGraphListener) تو فایل custom_listener.py داریم که کل بار تحلیل معنایی رو دوششه وضعیت داخلش هم اینطوره:

```
dict[id, {type, params, line}]:self.nodes •
list[{src, dst, label, line}]:self.edges •
dict[key, value]:self.config •
```

۱.۲.۴ انتشار مقدار در درخت

متدهای exitShape_expr و exitValue مقادیر رو روی خود context می‌نویسن:

قطعه کد ۲.۲.۴ > انتشار مقدار ctx.pvalue در exitShape_expr

```

1 def exitShape_expr(self, ctx):
2     ints = [int(ctx.INT_LITERAL(i).getText())
3             for i in range(ctx.INT_LITERAL().__len__())]
4     ctx.pvalue = tuple(ints)
5     ctx.ptype = tuple

```

این روش به ما اطمینان می‌دهد که وقتی از یک context می‌ایم بیرون، مقادیر بچه‌هاش دم دستمون باشن

۲.۲.۴ قوانین اعتبارسنجی

ما هشت تا قانون اعتبارسنجی داریم که تو دو تا فاز اجراشون می‌کنیم:

فاز اول (همون موقعی که داریم درخت رو بالا پایین می‌کنیم):

۱. شناسه تکراری گره: تو exitNode_decl چکش می‌کنیم

۲. ارجاع نامعتبر در یال: تو exitEdge_decl

۳. نوع لایه ناشناس: تو exitNode_decl

۴. عدم تطابق نوع پارامتر: بازم تو exitNode_decl

فاز دوم (وقتی گرافمون کامل شد تو exitGraph_block):

۵. گره خروجی اعلان نشده: تو _validate_output_declared

۶. گره یتیم: تو _validate_orphans مچش رو می‌گیریم

۷. آریتی residual != ۲: تو _validate_residual_arity

۸. تشخیص دور با kahn: تو _validate_no_cycles

۹. قابلیت دسترسی BFS: تو _validate_reachability

۳.۲.۴ قالب پیام خطا

قطعه کد ۳.۲.۴ > قالب خطا

```

1 [line L:C] SemanticError: <message>
2 [line L] ShapeError at '<node>': <message>

```

اینجا L همون شماره خطه و C هم ستون توکنمونه اگه به هر کدوم از این خطاها بخوریم، کامپایل متوقف می‌شه و با کد خروج ۱ می‌پریم بیرون

۴.۲.۴ FakeCtx برای خطاهای پس از پیمایش

خطاهایی که تو exitGraph_block پیش میان context زنده ندارن واسه همین ما یه کلاس جمع‌وجور به اسم FakeCtx ساختیم که شماره خط رو از گره برداره و با خودش بیاره:

قطعه کد ۴.۲.۴ > کلاس FakeCtx برای حفظ شماره خط

```
1 class _FakeCtx:
2     class start:
3         line    = info["line"]
4         column  = 0
```

۵.۲.۴ جریان اجرای شنونده

متد ParseTreeWalker.walk(listener, parse_tree) یه پیمایش عمق‌اول روی درختمون می‌زنه و قبل از اینکه بره سراغ بچه‌ها متد enterX(ctx) رو صدا می‌زنه و بعدش exitX(ctx) رو

تعریف ۵.۲.۴ > فراخوانی واکر در main.py

```
1 NNGraphCustomListener() = listener
2 ruleNames.parser = rule_names.listener
3 ParseTreeWalker() = walker
4 parse_tree), walk(listener.walker
```

اینجا شنونده سفارشی ما فقط متدهای *exit رو بازنویسی می‌کنه متدهای *enter رو هم همونجوری بدون تغییر از کلاس پایه NNGraphListener نگه می‌داریم

انتشار ویژگی‌ها (پایین به بالا)

ما مقادیر رو از طریق ویژگی‌های کانتکست می‌فرستیم بالا تو درخت:

قطعه کد ۶.۲.۴ > متد exitValue مقدار پارس‌شده را در ctx.pvalue ذخیره می‌کند

```
1 def exitValue(self, ctx):
2     if ctx.FLOAT_LITERAL():
3         ctx.pvalue = float(ctx.FLOAT_LITERAL().getText())
```

```

4         ctx.ptype = float
5     elif ctx.INT_LITERAL():
6         ctx.pvalue = int(ctx.INT_LITERAL().getText())
7         ctx.ptype = int
8     elif ctx.BOOL_LITERAL():
9         ctx.pvalue = ctx.BOOL_LITERAL().getText() == 'true'
10        ctx.ptype = bool
11    # ... other cases

```

اینطوری قوانین والد هم می‌تونن ویژگی‌های بچه‌هاشون رو بخونن:

قطعه کد ۷.۲.۴ > متد exitParam مقدار ctx.value().pvalue را از فرزند می‌خواند

```

1 def exitParam(self, ctx):
2     ctx.param_name = ctx.ID().getText()
3     ctx.param_value = ctx.value().pvalue
4     ctx.param_type = ctx.value().ptype

```

استخراج معنایی و شرح تفصیلی رفتار متدها

شنونده سفارشی ما با یه سری متد *exit کار می‌کنه که از پایین به بالا میان اینجا براتون باز می‌کنم که هر تابع شنونده دقیقاً چه کاری می‌کنه:

- **exitShape_expr(ctx)**: این میاد لیستی از ابعاد رو از توکن‌های INT_LITERAL پارس می‌کنه و یه tuple می‌سازه، بعدم می‌ریزه تو ctx.pvalue و نوعشم می‌ذاره tuple
- **exitValue(ctx)**: مقادیر لیتراال رو ارزیابی می‌کنه و تبدیلهشون می‌کنه به معادل پایتونی‌شون و می‌ریزه تو ctx.pvalue
- **exitModel_block(ctx)**: شناسه مدل شبکه رو از ctx.ID().getText() درمیاره و می‌ذاره تو self.model_name
- **exitInput_decl(ctx)**: شناسه متغیر ورودی رو تو self.input_id و شکل پارس‌شده‌ش رو تو self.input_shape ذخیره می‌کنه تازه یه شبه‌گره "__input__" هم می‌سازه واسه اعتبارسنجی
- **exitOutput_decl(ctx)**: شناسه خروجی رو نگه می‌داره تا بدونیم ته‌مون کجاست
- **exitParam(ctx)**: پارامترها رو هندل می‌کنه و شناسه و مقدار و نوعشون رو ذخیره می‌کنه
- **exitLayer_expr(ctx)**: پیکربندی‌های سازنده لایه رو می‌گیره و پارامترهاشون رو مپ می‌کنه

- **exitNode_decl(ctx)**: گره‌ها رو ثبت می‌کنه و چک می‌کنه شناسه تکراری نباشه و پارامترها هم به LAYER_SCHEMA بخورن
- **exitEdge_decl(ctx)**: یال‌ها رو پردازش می‌کنه و چک می‌کنه که مبدأ و مقصد تو `self.nodes` باشن و بعد یال رو ثبت می‌کنه
- **exitConfig_entry(ctx)**: تنظیمات سراسری مثل `device` یا `batch_size` رو می‌گیره می‌ذاره تو `self.config`
- **exitGraph_block(ctx)**: وقتی گراف کامل شد می‌پره وسط و اون پنج تا مرحله اعتبارسنجی که قبلاً گفتم رو اجرا می‌کنه
- **exitEveryRule(ctx)**: خروج از قوانین رو ردگیری می‌کنه و اگه لازم بود `make_ast_subtree` رو صدا می‌زنه
- **exitStart(ctx)**: کار درخت AST رو نهایی می‌کنه

الگوریتم‌های اعتبارسنجی گراف

متد `exitGraph_block` وقتی گراف ساخته شد این پنج تا کار رو می‌کنه:

۱. تشخیص گره یتیم (`_validate_orphans`):

قطعه کد ۸.۲.۴ > الگوریتم تشخیص گره یتیم

```

1 referenced = set()
2 for e in self.edges:
3     referenced.add(e["src"])
4     referenced.add(e["dst"])
5 for node_id, info in self.nodes.items():
6     if info["type"] == "__input__":
7         continue
8     if node_id not in referenced:
9         error(f"Node '{node_id}' is declared but never
           ↳ referenced.")

```

اینجا شناسه همه گره‌هایی که تو یال‌ها هستن رو جمع می‌کنیم هر گرهی که تعریف شده ولی تو این یال‌ها نیست، یتیم حساب می‌شه

۲. تشخیص دور (`_validate_no_cycles`):

اینجا رفتیم سراغ الگوریتم Kahn اگه بعد از پردازش، گرهی بمونه که `in_degree > 0` باشه، یعنی یه

جایی دور داریم

قطعه کد ۹.۲.۴ > الگوریتم Kahn برای تشخیص دور

```

1 in_degree = dict.fromkeys(self.nodes, 0)
2 adj       = {n: [] for n in self.nodes}
3 for e in self.edges:
4     adj[e["src"]].append(e["dst"])
5     in_degree[e["dst"]] += 1
6
7 queue     = deque(n for n, d in in_degree.items() if d == 0)
8 visited   = 0
9 while queue:
10    node = queue.popleft()
11    visited += 1
12    for nxt in adj[node]:
13        in_degree[nxt] -= 1
14        if in_degree[nxt] == 0:
15            queue.append(nxt)
16
17 if visited != len(self.nodes):
18     remaining = [n for n, d in in_degree.items() if d > 0]
19     error(f"Cycle detected involving nodes: {remaining}.")

```

۳. قابلیت دسترسی (`_validate_reachability`):

یه جستجوی BFS از گره ورودی می‌زنیم و انتظار داریم همه گره‌ها قابل دسترسی باشن

قطعه کد ۱۰.۲.۴ > بررسی قابلیت دسترسی با BFS

```

1 adj = {n: [] for n in self.nodes}
2 for e in self.edges:
3     adj[e["src"]].append(e["dst"])
4
5 visited = set()
6 queue   = deque([self.input_id])
7 while queue:
8     node = queue.popleft()

```



```

9     if node in visited:
10         continue
11     visited.add(node)
12     for nxt in adj[node]:
13         queue.append(nxt)
14
15 for node_id, info in self.nodes.items():
16     if node_id not in visited and info["type"] !=
17         ↪ "__input__":
18         error(f"Node '{node_id}' is not reachable from
19             ↪ input.")
20
21 if self.output_id not in visited:
22     error(f"Output node '{self.output_id}' is not
23         ↪ reachable.")

```

اینطوری بخش‌های جدا افتاده رو پیدا می‌کنیم

دقت کنین که تشخیص دور و قابلیت دسترسی دو تا چیز کاملاً جدان ممکنه گرافمون دور نداشته باشه ولی یه تیکه‌ش کلاً جدا باشه، یا همه جاش وصل باشه ولی دور داشته باشه ما واسه DAG شبکه‌مون به جفتش نیاز داریم

۳.۴ مولد کد

تعریف ۱.۳.۴ > CodeGenerator

ما کلاس CodeGenerator رو تو code_generator.py تو چهار تا گام زدیم:

۱. `_compute_graph()`: ساخت لیست‌های مجاورت و مرتب‌سازی توپولوژیک
۲. `_assign_vars()`: تخصیص نام متغیر پایتونی به هر گره
۳. `_emit_forward() + _emit_init()`: تولید خطوط کد
۴. `_assemble()`: هم‌زدن همه اینا با هم تو یه سورس نهایی پایتون

۴.۴ الگوریتم تخصیص متغیر

تعریف ۱.۴.۴ اصل اساسی تخصیص متغیر

ما $\text{var_at}[n] = 'x'$ رو فقط موقعی می‌ذاریم که $\text{out_degree}[\text{predecessor}] == 1$ باشه
وقتی یه گره قراره به چند جا بره ($\text{out_degree} > 1$)، یعنی شاخه‌های دیگه هنوز بهش نیاز
دارن اگه همونجا x رو بازنویسی کنیم که کارمون زاره واسه همین، گره بعدی همون اسم شناسه
خودش رو به عنوان متغیر می‌گیره

ما تو `_assign_vars` چهار تا قانون کلی داریم:

قطعه کد ۲.۴.۴ قوانین تخصیص متغیر در `_assign_vars`

```

1 Rule 1 - Input node:
2   var_at[input_id] = 'x'
3
4 Rule 2 - Merge node (in_degree > 1):
5   var_at[n] = 'x'           if out_degree[n] <= 1
6   var_at[n] = node_id      if out_degree[n] > 1
7
8 Rule 3 - Branch start (predecessor.out_degree > 1):
9   var_at[n] = node_id
10
11 Rule 4 - Linear continuation (predecessor.out_degree == 1):
12   var_at[n] = var_at[predecessor]
```

۱.۴.۴ ردیابی MLP (زنجیره خطی)

مثال ۳.۴.۴ ردیابی متغیر برای MLP

توپولوژی شبکه این شکلیه:

`x -> fc1 -> relu1 -> fc2 -> relu2 -> drop -> fc3 -> out`

اینجا هر گره قبلی $\text{out_degree} == 1$ داره، پس طبق قانون ۴ همیشه $\text{var_at} = 'x'$
می‌شه و تو forward همه‌جا از همون x استفاده می‌کنیم

۲.۴.۴ ردیابی ResBlock (فناوت سپس ادغام)

مثال ۴.۴.۴ ردیابی متغیر برای ResBlock

ما اینجا x رو هم به conv1 و هم به skip می‌فرستیم، پس $\text{out_degree}[x] = 2$ می‌شه

- واسه conv1 : قانون ۳ می‌گه $\text{var_at} = \text{'conv1'}$
- واسه بقیه: قانون ۴ می‌گه همه همون 'conv1' رو می‌گیرن
- واسه skip : قانون ۲ می‌گه $\text{var_at} = \text{'x'}$

اینم کدی که تولید می‌کنه:

قطعه کد ۵.۴.۴ pass forward تولیدشده برای ResBlock

```

1 conv1 = self.conv1(x)
2 conv1 = self.bn1(conv1)
3 conv1 = self.relu1(conv1)
4 conv1 = self.conv2(conv1)
5 conv1 = self.bn2(conv1)
6 x      = conv1 + x      # Residual node
7 x      = self.flatten(x)

```

۳.۴.۴ ردیابی InceptionBlock (دو شاخه موازی)

مثال ۶.۴.۴ ردیابی متغیر برای InceptionBlock

اینجا هم $\text{out_degree}[x] = 2$ هست

- واسه convA1 و convB1 : قانون ۳ بهمون 'convA1' و 'convB1' می‌ده
- واسه convA2 و convB2 : قانون ۴ رو داریم
- واسه cat1 : قانون ۲ دوباره بهمون 'x' می‌ده

کدش هم اینطوری درمیاد:

قطعه کد ۷.۴.۴ pass forward تولیدشده برای InceptionBlock

```

1 convA1 = self.convA1(x)
2 convA1 = self.convA2(convA1)
3 convB1 = self.convB1(x)
4 convB1 = self.convB2(convB1)
5 x      = torch.cat([convA1, convB1], dim=1)  # Concat
      ↪ node
6 x      = self.bn(x)
7 x      = self.out(x)

```

۴.۴.۴ ردیابی TransformerEncoder (دو residual ضمنی)

مثال ۸.۴.۴ ردیابی متغیر برای TransformerEncoder

اینجا وضعیت جالب‌تره، $out_degree[x] = 2$ و $out_degree[norm1] = 2$

- واسه $attn$: قانون ۳
 - واسه $drop1$: قانون ۴
 - واسه $norm1$: قانون ۲ بهمون 'norm1' می‌ده
 - واسه $ff1$: قانون ۳
 - واسه $gelu1$ و $drop2$ و $ff2$: قانون ۴
 - واسه $norm2$: قانون ۲ بهمون 'x' می‌ده
- خروجیش این شکلیه:

قطعه کد ۹.۴.۴ > pass forward تولیدشده برای TransformerEncoder

```

1  attn, _ = self.attn(x, x, x)      # MultiHeadAttn
   ↪ triple-pass
2  attn      = self.drop1(attn)
3  norm1     = self.norm1(x + attn)  # implicit residual_1
4  ff1       = self.ff1(norm1)
5  ff1       = self.gelu1(ff1)
6  ff1       = self.drop2(ff1)
7  ff1       = self.ff2(ff1)
8  x         = self.norm2(norm1 + ff1) # implicit
   ↪ residual_2
9  x         = self.out(x)
10 x         = self.flatten(x)
11 return x

```

۵.۴.۴ موارد خاص تولید کد

الگو	شرط	کد تولیدشده
MultiHeadAttn	نوع لایه	<code>out, _ = self.attn(x, x, x)</code>
LSTM / GRU	نوع لایه	<code>out, _ = self.lstm(x)</code>
Residual	بدون ماژول	<code>out = main + shortcut</code>
Add	بدون ماژول	<code>out = a + b</code>
Concat	بدون ماژول	<code>out = torch.cat([a,b], dim=d)</code>
Split	بدون ماژول	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>
Residual ضمنی	<code>label residual_*</code>	<code>out = self.norm(skip + main)</code>

نگاشت نام پارامتر: ما تو DSL اسما رو کوتاه کردیم ولی تو تولید کد به معادل‌های پایتونیشون برش گردوندیم مثلاً `in_ch` می‌شه `in_channels`

۶.۴.۴ ساختار کد خروجی

قطعه کد ۱۰.۴.۴ الگوی کد خروجی تولیدشده

```

1 import torch
2 import torch.nn as nn
3
4
5 class <ModelName>(nn.Module):
6     def __init__(self):
7         super(<ModelName>, self).__init__()
8         # one line per parameterized layer node
9
10    def forward(self, x):
11        # one line per node in topological order
12        return <output_var>
13
14
15 if __name__ == '__main__':
16     device = torch.device('<device>')
17     model = <ModelName>().to(device)
18     x = torch.randn(<batch_size>,
19                    ↪ <input_shape>).to(device)
20     print(model(x).shape)

```

۵.۴ استنتاج شکل

تعریف ۱.۵.۴ inference shape

ما ماژول `shape_inference.py` رو نوشتیم که شکل تنسور رو از ورودی تو کل گراف پخش می‌کنه قوانینش هم سادس، مثلاً Linear بعد آخر رو عوض می‌کنه یا Flatten ابعاد رو با هم یکی می‌کنه

اگه ابعاد به هم نخورن، خیلی شیک خطای `ShapeError` می‌دیم و کامپایل رو قطع می‌کنیم با پرچم `--show-shapes` - هم می‌شه شکل‌ها رو دید

۶.۴ تجسم Graphviz

تو ماژول `graph_viz.py` ما خروجی Graphviz DOT می‌دیم بیرون به نظم رنگ‌بندیشم باحاله، مثلاً ورودی سبزه و لایه‌های نرمال‌سازی بنفش گره خروجی رم با دو تا خط مشخص کردیم و یال‌های برچسب‌دار رو خط‌چین زدیم

قطعه کد ۱.۶.۴ > تولید و رندر نمودار

```
1 python main.py -i input/inception.nng --graph-viz
  ↳ output/inception.dot
2 dot -Tpng output/inception.dot -o output/inception.png
```

۷.۴ داربست آموزش

اگه `loss` رو تو تنظیمات بدیم، کد ژنراتورمون یه حلقه آموزش مشتی برامون سرهم می‌کنه توابع خطای معروف مثل `CrossEntropyLoss` و `MSELoss` و بهینه‌سازی مثل `Adam` و `SGD` رو هم پشتیبانی می‌کنیم

۸.۴ حاشیه‌نویسی کوانتیزاسیون

پارامتر سراسری `quant` رو همه‌جا می‌شه استفاده کرد سه تا حالت `"dynamic"` و `"static"` و `"qat"` رو ساپورت می‌کنیم اگه کوانتیزاسیون فعال باشه، اون `QuantStub` و دوستاشم به کدمون اضافه می‌شن

۹.۴ صادرکردن ONNX

ماژول `onnx_export.py` مدلمون رو نمونه‌سازی می‌کنه و یه خروجی تمیز ONNX می‌ده که با محور دسته‌ای پویاست

قطعه کد ۱.۹.۴ > صادرکردن مدل به ONNX

```
1 python main.py -i input/mlp.nng --onnx output/mlp.onnx
```

فصل ۵

ارزیابی

۱.۵ مجموعه آزمایش

تعریف ۱.۱.۵ • آمار آزمایش

- تعداد کل تست‌ها: ۵۵
- فریم‌ورک: pytest نسخه ۲.۴.۸
- زمان اجرا: حدود ۹.۱ ثانیه

فایل	تست‌ها	پوشش
test_parser.py	۸	گرامر بدون خطای نحوی برای ۴ نمونه؛ config، پرچسب، شکل
test_listener.py	۱۴	۴ ورودی معتبر؛ ۱۰ شرط خطا با SystemExit
test_codegen.py	۱۹	نام کلاس‌های nn، مقادیر پارامتر، الگوهای forward
test_e2e.py	۱۴	کامپایل کامل + ایجاد نمونه + for- pass ward با شکل صحیح

اینم دستورش:

قطعه کد ۲.۱.۵ > اجرای مجموعه آزمایش

```
1 python -m pytest tests/ -v
```

۲.۵ نتایج pass forward

مثال ۱.۲.۵ > MLP

- ورودی: $\text{tensor}(784)$ – خروجی: $(\text{batch}, 10)$
- آزمایش: softmax ردیف‌ها به ۱ جمع می‌شن

مثال ۲.۲.۵ > ResBlock

- ورودی: $\text{tensor}(64, 32, 32)$ – خروجی: $(\text{batch}, 65536)$
- آزمایش: شکش درسته و تو حالت eval خروجی قطعی

مثال ۳.۲.۵ > InceptionBlock

- ورودی: $\text{tensor}(32, 56, 56)$ – خروجی: $(\text{batch}, 48, 56, 56)$
- شاخه A با ۱۶ و شاخه B با ۳۲ کانال ادغام می‌شن و ۴۸ کانال می‌دن

مثال ۴.۲.۵ > TransformerEncoder

- ورودی: $\text{tensor}(512, 128)$ – خروجی: $(1024, 128)$
- دو تا زیرلایه residual ضمنی داریم

۳.۵ اثبات درستی متغیرها

تعریف ۱.۳.۵ > اثبات عدم بازنویسی تنسور

- ما واسه هر نمونه، $\text{data_ptr}()$ تنسور ورودی رو چک کردیم:
 - **ResBlock**: اینجا $\text{data_ptr}()$ دست‌نخورده مونده
 - **Inception**: تو جفت شاخه‌ها $\text{data_ptr}()$ یکسانه
 - **Transformer**: تو تمام مسیرهای residual هم مقادیر تغییر نکردن
- این دقیقاً همون چیزیه که ما از قانونمون انتظار داشتیم

۴.۵ آمار کدبیس

خطوط	فایل/دایرکتوری
۵۶	NNGraph.g4
۳۰۳	custom_listener.py
۳۲۷	code_generator.py
۱۹۳	shape_inference.py
۹۰	graph_viz.py
۴۷	onnx_export.py
۱۲۱	main.py
۵۷۵	tests/ (۴ فایل)
۱۰۰≈	required_code_collection/
۱۸۱۲≈	جمع

- زبان: Python ۳.۱۴.۳
- ANTLR: ۴.۱۳.۲
- PyTorch: ۲.۱۰.۰
- pytest: ۸.۴.۲

فصل ۶

تصمیمات طراحی

۱.۶ Listener در برابر Visitor

الگوی Listener خیلی راحت بهمون اجازه داد وضعیت رو نگه داریم و مستقیم رو `self.nodes` و `self.edges` کار کنیم

۲.۶ `load_from_listener()` در برابر `generate(traversal)`

واسه کامپایلر گراف، ساختار درختی و گراف خودش خیلی طبیعی‌تر از یه لیست خطی توکنه، واسه همین `load_from_listener` رو ترجیح دادیم

۳.۶ نام‌گذاری متغیر با `node_id`

با اینکه می‌شد از اسمای تک‌حرفی استفاده کرد، ما رفتیم سراغ شناسه واقعی گره چون هم قطعی، هم خیلی خواناتر می‌شه و تو گرافای بزرگ هم کارمون رو راحت می‌کنه

فصل ۷

نتیجه‌گیری و کارهای آینده

۱.۷ نتیجه‌گیری

در کل ما به NNGraph DSL مشتی با شش مرحله کامپایل ساختیم به کامپایلر با ۱۴ قانون تجزیه‌گر، ۲۴ نوع لایه و کلی قوانین اعتبارسنجی داریم که حتی پیچیده‌ترین شاخه‌ها رو هم بدون باگ هندل می‌کنه تستاشم رو ۴ تا معماری کامل ران کردیم و جواب داد

۲.۷ قابلیت‌های پیاده‌سازی‌شده

قابلیت	توضیح
استنتاج شکل	انتشار شکل تنسور و تشخیص عدم تطابق در کامپایل (- - show-shapes)
تجسم Graphviz	خروجی نمودار گراف مدل به فرمت DOT (- - graph-viz)
داربست آموزش	تولید حلقه آموزش با بهینه‌ساز و تابع خطا از بلوک config
کوانتیزاسیون	متادیتا quant برای تبدیل مدل به نسخه‌های کوانتیزه
صادرکردن ONNX	صادرکردن مدل به فرمت ONNX (- - onnx)

۳.۷ کارهای آینده

قابلیت توضیح	
زیرگراف‌های نام‌دار	بلوک‌های ماکرو قابل استفاده مجدد
کنترل جریان پویا	شاخه‌های <code>else/if</code> درون <code>forward</code>

منابع

۱. Parr, T. (۲۰۱۳.) The Definitive ANTLR ۴ Reference. Pragmatic Bookshelf.
۲. Paszke, A., et al. (۲۰۱۹.) PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS ۲۰۱۹.
۳. NADER Framework: Neural Architecture Design via Representation (referenced in NNGraph_DSL_Specification.md.)
- Zhang, A. M. amznotes LaTeX template. github.com/alexmingzhang/latex-notes-template